

Полная инструкция: от идеи до автономного Telegram-бота

Пошаговое руководство: как создать Telegram-бота и разместить его на бесплатном сервере так, чтобы он работал 24/7 сам по себе, независимо от вашего компьютера. Инструкция написана по следам реального прохождения всего пути — включая все ошибки, на которые мы наступили, и как их обойти сразу, не повторяя их.

Оглавление

1. Что нужно заранее
 2. Создание бота в Telegram
 3. Код бота
 4. Локальный запуск на Windows (для разработки)
 5. Создание бесплатного сервера в Oracle Cloud
 6. Подключение к серверу по SSH
 7. Установка Python и окружения на сервере
 8. Разворачивание бота на сервере
 9. Автозапуск через systemd
 10. Шпаргалка команд на каждый день
 11. Сводка всех проблем и решений
-

1. Что нужно заранее

- Компьютер с Windows
 - Аккаунт Telegram
 - Email для регистрации в Oracle Cloud (плюс банковская карта — для верификации личности, деньги с бесплатного тарифа не списываются)
 - GitHub-аккаунт, если код бота хранится там
-

2. Создание бота в Telegram

1. В Telegram напишите **@BotFather**
2. Отправьте `/newbot`, следуйте инструкциям (имя бота, username)

3. BotFather пришлёт **токен** вида `123456789:AAExampleTokenFromBotFather` — сохраните его, он понадобится дальше. Токен — секретная вещь, как пароль: у кого он есть, тот полностью управляет ботом.
4. (Опционально, но полезно) В BotFather откройте **Bot Settings → Inline Mode**, включите и задайте текст-подсказку — это позволит вызывать бота из любого чата командой `@ИмяБота текст`, без добавления бота в этот чат.
5. Узнайте свой **Telegram user ID** через [@userinfobot](#) — пригодится, если в боте есть команды только для владельца (например, рассылка).

3. Код бота

Если код уже есть в GitHub-репозитории — просто клонируйте его (см. раздел 8). Если создаёте с нуля — структура проекта (на примере этого бота):

```
blender_bot/  
├── bot.py           # точка входа, регистрация обработчиков  
├── config.py        # переменные окружения, пути к данным  
├── requirements.txt  
├── .env.example     # шаблон, без реальных секретов  
├── .gitignore       # исключает .env, venv/, кэш  
├── data/            # JSON с контентом бота  
├── handlers/        # обработчики команд/сообщений  
└── utils/           # вспомогательная логика (поиск, переводчик и т.д.)
```

Ключевые уроки по коду, если пишете похожего бота:

- Храните токен и другие секреты только в `.env` (добавленном в `.gitignore`), никогда не коммитьте их в git
- Файлы, которые бот создаёт сам во время работы (логи, список подписчиков) — тоже добавляйте в `.gitignore`, они не должны попадать в репозиторий
- Если бот использует библиотеку `python-telegram-bot`, указывайте в `requirements.txt` **минимальную**, а не жёстко зафиксированную версию (`python-telegram-bot>=21.6`, а не `==21.6`) — так на новых версиях Python всегда подтянется совместимая сборка (см. проблему №1 в разделе 11)

4. Локальный запуск на Windows (для разработки)

1. Установите **Python** с [python.org](#) — при установке обязательно отметьте "**Add python.exe to PATH**"

⚠ **Если у вас Windows поставила совсем свежую версию Python (например 3.14)** — заранее знайте: некоторые библиотеки (в частности `python-telegram-bot`) могут упасть с ошибкой `RuntimeError: There is no current event loop`. Решение — либо `pip install --upgrade python-telegram-bot` (часто помогает), либо поставить дополнительно Python 3.12 через `py -3.12` и создавать `venv` именно на нём. Подробности — проблема №1 в разделе 11.

2. Откройте терминал в папке проекта: в Проводнике зайдите в папку, в адресной строке наберите `cmd`, нажмите Enter
3. Создайте виртуальное окружение и установите зависимости:

```
python -m venv venv venv\Scripts\activate pip install -r requirements.txt
```

4. Скопируйте `.env.example` в `.env`, впишите реальный токен
5. Запустите:

```
python bot.py
```

Строка `Бот запущен` в терминале — значит всё работает. Проверьте в Telegram командой `/start`.

Это годится для разработки и тестов, но бот работает только пока открыт терминал. Дальше — как разместить его на сервере, чтобы он работал сам по себе.

5. Создание бесплатного сервера в Oracle Cloud

Используем **Oracle Cloud Free Tier** — там есть по-настоящему бесплатный навсегда сервер (Always Free), которого достаточно для одного небольшого бота.

5.1 Регистрация

1. Зайдите на oracle.com/cloud/free → **Start for free**
2. Зарегистрируйтесь (email, телефон, карта для верификации — не списывает деньги с Always Free ресурсов)

5.2 Создание сервера (Compute Instance)

В консоли: ☰ меню → **Compute** → **Instances** → **Create Instance**.

Basic information: - Name — можно оставить автоматическое, роли не играет - Image — нажмите **Change image**, выберите **Ubuntu** (последняя LTS-версия, например 20.04 или 24.04)

Shape: - Нажмите **Change shape**, выберите вариант с пометкой **Always Free-eligible** (обычно `VM.Standard.E2.1.Micro`)

Security: - Ничего не меняйте, оставьте значения по умолчанию (Shielded instance выключен)

Networking — самый проблемный шаг, читайте внимательно:

1. **Primary network** → переключите на **Create new virtual cloud network** (если у вас ещё нет ни одной сети — это нормально для первого сервера)
2. **Subnet** → переключите на **Create new public subnet** (обязательно **public**)
3. Прокрутите вниз до **Public IPv4 address assignment** → включите **Automatically assign public IPv4 address**

⚠ Известный баг: этот переключатель у нас несколько раз не включался (оставался серым), несмотря на то, что public subnet был выбран верно. Это не ошибка пользователя, а глюк интерфейса Oracle. **Если не включается — не тратьте на это время, создавайте сервер как есть, без публичного IP.** Добавим его отдельно после создания сервера (шаг 5.4 ниже) — это работает надёжнее, чем сам мастер создания.

Add SSH keys: - Оставьте выбранным **Generate a key pair for me** - Нажмите **Download private key** — скачается файл `.key`. **Сохраните его в отдельную, легко находимую папку** (например `C:\oracle-vps\`) — без него доступ к серверу восстановить нельзя

Storage: - Ничего не меняйте, оставьте значения по умолчанию

Нажмите **Create**. Через 1-2 минуты статус сервера станет **Running**.

5.3 Если публичный IP не появился при создании

На странице сервера откройте вкладку **Сети (Networking)**. Если поле **Общедоступный адрес IPv4** пустое — там же должен быть блок **Экспресс-действия → «Подключение общедоступной подсети к Интернету»** с кнопкой **Подключение**. Нажмите её — Oracle подключит интернет-шлюз и настроит таблицу маршрутизации автоматически.

Если экспресс-действия нет или не помогло — добавьте IP вручную:

1. Вкладка **Сети** инстанса → нажмите на имя VNIC
2. Вкладка **Администрирование IP-адресов**
3. Найдите приватный адрес → значок `:` (три точки) → редактировать
4. **Тип общедоступного IP-адреса** → выберите **Динамический общедоступный IP-адрес** → **Обновить**

После этого на вкладке инстанса появится публичный IP — запишите его, он понадобится для подключения по SSH.

6. Подключение к серверу по SSH

1. В Проводнике откройте папку со скачанным `.key`-файлом, в адресной строке наберите `cmd`
2. Подключитесь:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_ПУБЛИЧНЫЙ_IP
```

(пользователь именно `ubuntu`, если ставили образ Ubuntu)

3. На вопрос про fingerprint ("Are you sure you want to continue connecting?") ответьте `yes`

⚠ Частая ошибка на Windows: UNPROTECTED PRIVATE KEY FILE. Windows по умолчанию даёт слишком открытые права на файл ключа. Лечится так (в обычном Windows-терминале, не на сервере):

```
icacls ssh-key-ИМЯФАЙЛА.key /inheritance:r icacls ssh-key-ИМЯФАЙЛА.key /grant:r "%username%":R
```

Если после этого ошибка осталась — проверьте, что реально удалось убрать:

```
icacls ssh-key-ИМЯФАЙЛА.key
```

Если в списке остались `BUILTIN\Administrators`, `NT AUTHORITY\SYSTEM`, `Authenticated Users` или `BUILTIN\Users` — уберите их явно:

```
icacls ssh-key-ИМЯФАЙЛА.key /remove:g "BUILTIN\Administrators" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "NT AUTHORITY\SYSTEM" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "NT AUTHORITY\Authenticated Users" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "BUILTIN\Users"
```

Должна остаться только ваша учётная запись. Пробуйте подключаться заново.

⚠ Если SSH неожиданно обрывается (`client_loop: send disconnect: Connection reset`) — это иногда бывает, особенно при первом подключении. Просто подключайтесь заново той же командой `ssh -i ...`.

Если увидели приглашение вида `ubuntu@instance-...:~$` — вы внутри сервера.

7. Установка Python и окружения на сервере

7.1 Обновление системы и git

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git
```

7.2 Установка Python 3.11 через pyenv

Проверьте версию системного Python:

```
python3 --version
```

Ubuntu 20.04 обычно ставит Python 3.8 — этого мало для современных версий `python-telegram-bot` (нужен минимум 3.9-3.10, лучше 3.11).

⚠ Не тратьте время на deadsnakes PPA. Стандартный совет в интернете — поставить нужную версию Python через `add-apt-repository ppa:deadsnakes/ppa`. У нас этот путь трижды ломался (пакет "находился" в репозитории, но `apt install` не мог его увидеть — кэш apt не докачивал список пакетов). Чинится через `sudo rm -rf /var/lib/apt/lists/* && sudo apt update`, но результат нестабилен. **Сразу используйте pyenv** — он не зависит от репозитория Ubuntu и просто компилирует Python из исходников.

Установите инструменты для сборки и сам pyenv:

```
sudo apt install -y build-essential zlib1g-dev libncurses5-dev libgdbm-dev libnss3-dev
sudo apt install -y screen
curl -fsSL https://pyenv.run | bash
```

Добавьте pyenv в переменные окружения — **одной командой**, не по одной строке через несколько `echo`:

⚠ Если писать три отдельные команды `echo '...' >> ~/.bashrc` одну за другой — при вставке в терминал строки могут "слипнуться" без переноса (например, `~/.bashrc echo`), и `.bashrc` испортится с ошибкой синтаксиса при следующем открытии терминала. Используйте один `printf`:

```
printf '\nexport PYENV_ROOT="$HOME/.pyenv"\n[[ -d $PYENV_ROOT/bin ]] && export PATH="$P
source ~/.bashrc
pyenv --version
```

Если `.bashrc` всё же оказался повреждён (ошибка `syntax error near unexpected token` при `source`), почистите дубликаты и пересоздайте блок:

```
sed -i '/PYENV_ROOT/d; /pyenv init/d; /pyenv virtualenv-init/d' ~/.bashrc
printf '\nexport PYENV_ROOT="$HOME/.pyenv"\n[[ -d $PYENV_ROOT/bin ]] && export PATH="$P
source ~/.bashrc
```

7.3 Компиляция Python — обязательно внутри `screen`

⚠ Компиляция Python на слабом сервере занимает **10-15 минут**. Если за это время оборвётся SSH-соединение (а на первых порах оно обрывается часто) — процесс компиляции **умрёт вместе с закрытым терминалом**, и всё придётся начинать заново. Чтобы это не повторилось — обязательно запускайте установку внутри `screen`, программы, которая держит процессы живыми на сервере независимо от вашего SSH-окна.

```
screen -S pyenv_install  
pyenv install 3.11.9
```

Дождитесь строки `Installed Python-3.11.9 to /home/ubuntu/.pyenv/versions/3.11.9`.

Если SSH оборвётся во время ожидания — просто переподключитесь и вернитесь к сессии:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_IP  
screen -r pyenv_install
```

Если пишет `There is no screen to be resumed... (Attached)` — добавьте флаг `-d`, который принудительно отключает старое мёртвое подключение:

```
screen -d -r pyenv_install
```

После успешной установки:

```
pyenv global 3.11.9  
python3 --version
```

Должно показать `Python 3.11.9`. Выйдите из `screen` обычной командой `exit`.

8. Разворачивание бота на сервере

8.1 Скачать код

```
git clone -b ИМЯ_ВЕТКИ https://github.com/ВАШ_АККАУНТ/ВАШ_РЕПОЗИТОРИЙ.git  
cd ВАШ_РЕПОЗИТОРИЙ/blender_bot
```

8.2 Виртуальное окружение и зависимости

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

В начале строки терминала должно появиться `(venv)`.

8.3 Настройка `.env`

```
nano .env
```

Впишите (замените на реальные значения):

```
BOT_TOKEN=ваш_настоящий_токен_от_botfather
OWNER_ID=ваш_telegram_id
```

Сохраняйте строго в этом порядке:

1. **Ctrl+O**, затем **Enter** (сохранить)
2. Только затем **Ctrl+X** (выйти)

⚠ Если закрыть терминал/окно раньше, чем нажать Ctrl+O → Enter → Ctrl+X — файл не сохранится. **Обязательно проверьте после сохранения:**

```
cat .env
```

Должны увидеть обе строки. Если `cat: .env: No such file or directory` — значит не сохранилось, повторите `nano .env` заново.

⚠ Если при повторном открытии nano пишет `File .env is being edited by ubuntu (with nano ..., PID ...); open anyway?` — это безобидное предупреждение об оставшемся "зависшем" процессе от прошлой попытки. Смело жмите **Y**.

8.4 Тестовый запуск

```
python bot.py
```

Строка `Бот запущен` и `Application started` — значит всё работает. Проверьте в Telegram: `/start`. Остановите тестовый запуск клавишами **Ctrl+C** перед следующим шагом.

9. Автозапуск через systemd

Это финальный шаг, который делает бота по-настоящему автономным — он будет работать даже после закрытия терминала и перезагрузки сервера.

```
sudo nano /etc/systemd/system/blenderbot.service
```

Вставьте (замените пути на свои, если отличаются — узнать текущий путь можно командой `pwd`, находясь в папке бота):

```
[Unit]
Description=Blender Telegram Bot
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot
ExecStart=/home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot/venv/bin/python /home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot/main.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Сохраните: **Ctrl+O** → **Enter** → **Ctrl+X**. Проверьте содержимое (`cat /etc/systemd/system/blenderbot.service`) — так же, как с `.env` .

Включите и запустите службу:

```
sudo systemctl daemon-reload
sudo systemctl enable blenderbot
sudo systemctl start blenderbot
sudo systemctl status blenderbot
```

Должно быть **Active: active (running)** и **enabled**. Готово — можно закрывать терминал и выключать компьютер, бот продолжит работать.

10. Шпаргалка команд на каждый день

Подключение к серверу:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_IP
```

Статус бота:

```
sudo systemctl status blenderbot
```

Перезапуск (например, после обновления кода):

```
sudo systemctl restart blenderbot
```

Остановка:

```
sudo systemctl stop blenderbot
```

Живые логи бота:

```
sudo journalctl -u blenderbot -f
```

(выход — Ctrl+C)

Обновление кода бота на сервере:

```
cd ~/ВАШ_РЕПОЗИТОРИЙ/blender_bot
git pull origin ИМЯ_ВЕТКИ
sudo systemctl restart blenderbot
```

11. Сводка всех проблем и решений

№	Проблема	Причина	Решение
1	<code>RuntimeError: There is no current event loop</code> при запуске бота на Windows	Слишком новая версия Python (3.14) несовместима с библиотекой	<code>pip install --upgrade python-telegram-bot</code> , либо <code>venv</code> на Python 3.12
2	Переключатель "Automatically assign public IPv4 address" не включается при создании сервера	Известный глюк интерфейса Oracle Cloud	Создать сервер без IP, добавить публичный IP отдельно через вкладку "Сети" уже созданного инстанса
3	<code>UNPROTECTED PRIVATE KEY FILE</code> при SSH с Windows	Слишком открытые NTFS-права на файл ключа	<code>icacls</code> — убрать все права кроме своей учётной записи (см. раздел 6)
4	<code>E: Unable to locate package python3.11</code> из <code>deadsnakes PPA</code>	Повреждённый/недокачанный кэш <code>apt</code>	Использовать <code>pyenv</code> вместо PPA (см. раздел 7.2)

№	Проблема	Причина	Решение
5	Процесс компиляции Python обрывается при разрыве SSH	Процесс без защиты завершается вместе с закрытым терминалом	Запускать долгие операции внутри <code>screen</code>
6	<code>screen -r</code> пишет "no screen to be resumed... (Attached)"	Screen-сессия осталась помечена как подключённая к мёртвому терминалу	<code>screen -d -r ИМЯ_СЕССИИ</code> (флаг <code>-d</code> принудительно отключает старое)
7	Синтаксическая ошибка в <code>.bashrc</code> при <code>source ~/.bashrc</code>	Несколько отдельных <code>echo</code> -команд "слиплись" при вставке в терминал	Добавлять переменные одной командой <code>printf</code> , не несколькими <code>echo</code>
8	<code>.env</code> или файл <code>systemd-службы</code> не сохраняется в <code>nano</code>	Терминал закрыт раньше, чем завершена последовательность сохранения	Строго <code>Ctrl+O</code> → <code>Enter</code> → <code>Ctrl+X</code> , затем проверять <code>cat</code>

Составлено по итогам реального разворачивания Blender-бота на Oracle Cloud Free Tier. Все описанные проблемы — не гипотетические, а реально встреченные и решённые в процессе.